Network computing

Indice

Data centers

We have 2 main types of traffic inside the data center network:

1. **North-south**: requests come from the user and traverse in depth the network
2. **East-West**: computations that send requests intra-data center

The **majority of traffic inside a center is east-west**. The implication of this is that we need to have **any-to-any communication** that needs to happen at full speed, which requires a **high bandwidth**, and a consistent low latency, this means that the **worst case (tail) latency needs to be low**.

At a birds-eye view, a data center is just a massive switch offering billions of network ports to different users. The practical approach is to have a **network of switches (fabric)**.

Traditional tree-like topology

Originally data center is composed of **multiple racks of servers**. On top of each rack there is a **top-of-rack switch. ToR-switches are in turn connected to aggregation-switches. Aggregation-switches are then connected to the core-switches that connect the network to the outside**. An **aggregation of racks** was called a **pod**. **Connections between servers-tor-aggregation switches is L2 and is called edge layer; connections between aggregate and core switches are on L3 and is called core layer**.

L2 vs L3 switches have different advantages and disadvantages:

1. **L2 has fixed IP addresses** and have **auto-configuration** but **scale worse**;
2. **L3 routing** is more scalable through **hierarchical addressing**. We can also have **multipath routing** through equal cost multipath. **However, they require more configuration** and **migrations** would be problematic since they **would require IP address changes**.

A few definitions:

1. **Diameter**: max number between any two points;
2. **Bisection bandwidth**: if the network would be bisected, the bisection bandwidth of the topology is the bandwidth available between the two partitions;
3. **Full bisection bandwidth**: the ability for all hosts in one half to simultaneously talk at full speed to all host of the other half.

Scaling up the traditional tree-topology is very costly since going up the tree we require higher and higher bandwidth.

A band-aid is **over-subscribing**: we **provision less than the full bandwidth** for the hosts. We can define over-subscription as the **ratio of the worst-case achievable aggregate bandwidth among the end hosts to the total bisection bandwidth of a particular communication topology**. Over-subscription easily baloons as we go up the tree.

Fat tree topology

We have two approaches:

1. scale-up: stick to the tree and simply buy bigger links
2. scale-out: scale for bigger amount of hosts

Some **noteworthy topologies** are: the **fat-tree, bcube and jellyfish. The most-standard topology is the fat-tree**.

The **fat-tree** is a special type of **Clos network**: a type of non-blocking, multistage switching architecture that reduces the number of ports required in an interconnected fabric.

A **fat-tree is characterized by a parameter** k. A k-ary fat-tree has a three layer topology with:

- Each pod consists of $(k/2)^2$ servers and 2 layers of $k/2$ k-port switches;
- Each edge switch connects to $k/2$ servers and $k/2$ aggregate switches;
- Each aggregate switch connect to $k/2$ edge and $k/2$ core switches
- We have $(k/2)^2$ core switches that connect to k pods.

We have some **interesting properties**:

1. **Identical bandwidths at any bisection**
2. **Each layer has the same aggregated bandwidth**
3. Can be **built using cheap devices at uniform capacity**
4. **Scalability is very good** (a k-port switch supports $k^3/4$ servers).

The fat-tree brings **additional problems**: we **need a routing protocol that exploits all paths** and **transport protocol must fill all pipes**.

Flows

Flows in data centers have different characteristics: **we have applications that generate bursty flows, large flows or short flows**. This implies that our fabric needs to **satisfy two different constraints**:

1. **Short messages require low latencies**
2. **Large flows require high throughput**

This two styles of flows **interfere with each other**: we need **intelligent routing** to avoid that large-flows clash with small flows to fulfill our requirements.

The problem is, however, not only about routing: let's say we have data striped across different servers. We might incur in a problem called **TCP Incast**: we cram n flows into only one link, resulting into packet loss and triggering TCP shenanigans. Solving this problem **requires smarter congestion control**.

Costs

Total costs of a data center can be upwards of 0.25 B$ for mega-data centers. Server costs dominate, but network costs are also predominant. We work with long provisioning scales, where new servers are purchased quarterly at best.

Server costs include:

1. Uneven application fit: most application exhaust only on resource on the server, stranding the others;
2. Uncertainty in the demand of a service (spikes);
3. Risk management.

Google jupiter's topology

Google started from a **single commercial switching asic** (to keep costs down), and combined them into a single more powerful chip (**chiplet-style design**).

Software-defined networking

Originally we had **two devices**:

1. The **switch** routes packets only based on **layer 2** data

2. The **router** routes packets only based on **layer 3** data

Internally, a **router** is usually composed of **two main components**: a **CPU running an OS** (with all various router features like routing protocols and tables) and an **asic that does the actual routing**. The asic works with data given by the OS driver via a forwarding table.

When a **packet enters a router**, we **parse the header** and then decide how to handle it:

1. If it is a **data packet**, it **stays on the asic and uses the forwarding table** to get sent fast
2. If it a control **layer packet**, it gets **sent to the router OS** for slower processing

We can do this since **control layer updates are much rarer than data**.

What if we need to run our network on a **routing protocol not yet supported by our router**? We could possibly request an update from the vendor, but this is slow and not very flexible.

The basic idea of **SDN** is to **separate even more the control from the data plane**: the two layers are even **separated physically**. The **control plane** is now a **remote machine** that is connected to the data-plane via a communication channel. Since we do not need to change the data plane often, we can have a **simple API** to it and then implement what we want in the control plane.

This architecture lets us have **many packet-forwarding devices and a single network OS with user-controlled features**. To do this **we need**:

1. A **network OS** that provides the necessary abstractions
2. **Consistent, up-to-date global network view**
3. Open **interface to packet forwarding**

We can then **write programs that take the global network view as input and outputs a configuration of each of the network devices**. The **interface between OS and programs is the *northbound interface***, while the **interface between OS and the packet-forwarding devices is the *southbound interface***.

OpenFlow

OpenFlow is **realization of SDN**. It is a **set of protocols and APIs**. The CPU of an openflow switch runs an openflow daemon that handles communication with the server.

A program can do a variety of things such as adding/removing flow rules at switches, collect statistics and establishing connectivity at switches.

The **default** behaviour (called **reactive**) is the following: **when a packet arrives at the switch try to match it to the rules, if a match can be found it is processed accordingly, otherwise processing is interrupted and the header is sent to the controller; the controller then processes the header and sends a new ad-hoc rule**.

The default behaviour, or **mode**, has the advantage of having **high control granularity and making efficient use of flow tables**. However it incurs in **additional setup costs for every new flow**. Moreover **if the connectivity to the controller is lost, the switch could become a paperweight**.

Complementing the default mode, another one (called **proactive**) exists: **instead of interrupting the flow of packets, at setup time the controller pre-populates all possible rules (using wildcards to match every possibility)**.

This second mode **exchanges granularity for speed**. However we have zero additional setup costs and our switch is always up, even in case of loss of connection to the controller.

Is an openFlow switch a switch or a router: it depends. The flexibility enables, however, **flexibility in the control plane level**. We can also have a legacy compatibility mode with other control planes protocols.

The **problem** is that **openFlow assumes that most network switches do the same things** and that switches **have a fixed, well-known behaviour**. Instead of **repeatedly extending the standard** we can define a **flexible mechanism for parsing arbitrary packets and matching header fields through a common interface**.

p4

Usually network systems are configured bottom-up. However, how we did in all other branches of IT, the more flexible approach is to **do things top-down**. This means we need **programs to compile and execute directly on the switch, that now becomes programmable**.

p4 is an **attempt at bringing to networking what RISC brought to computing**.

The most important **aspects to consider** are:

1. **Chip speed**: **we can now make programmable switches chips as fast as fixed ones**
2. **Chip complexity**: there are **too many protocols to correctly hard-code in silicon**
3. **Chip technology**: the **difference in chip area and power between programmable and fixed function is going away**

On these types of chips, **serial IO is massive**: almost 30% of the area of the chip. Due to **Moore's law**, **each generation we halve the size of the serial IO chip**. If we maintain the **area the same, we can double the speed of the IO**.

The **remaining part** that does not do IO is **divided between packet processing (50%) and buffers/queues (20%)**. Of the **packet processing logic, half is only memories while the rest is actual logic**. Like with serial IO, **we can double the memory each generation while maintaining the same area surface**; **however** with logic **we do not implement new logic every generation, meaning that the logic part shrinks every year**.

Since only the logic part is the one differentiating between programmable and non-programmable switches, **the part that actually makes a difference is become smaller each generation**.

P4 is **not openFlow 2.0, it addresses a much more general problem**: **programming the dataplane**; openFlow is just a system for dynamically enabling/disabling rules. **With p4 it is possible to implement an openflow switch**, for instance.

p4 programs are **built against a p4 architecture model**. This makes it possible for the same program to compile for different targets. **The binary file generated by the compiler gets loaded onto the switch**. The **program effectively creates a new "control plane" that interacts with the data plane through an API**.

PISA (Protocol independent switch architecture)

We have **a set of identical pipeline stages**. Each of them is **designed to perform a set of match-action operations that can be done in parallel**. **A programmable parser extracts the headers from the incoming packet** and **feeds them to the pipeline**.

A **packet header vector** (PHV) is **used as a container that hosts the match keys for the match+action units**. Based on match entries, **we can perform some actions that can change/add/remove packet headers**. This is **done for each stage** of the pipeline. Finally, the **packet gets reassembled in the deparser**.

To add even more flexibility, the **pipeline outputs into a traffic manager**, that does the switching. This manager **then feeds directly into another pipeline that can modify the packet, but not change its switching**. This way **we have pre-switching (ingress) and post-switching (egress) processing**, enabling us to also do matching on the destination.

Each pipeline stage is composed of different match logic units (associative memories that match headers against rules) **and action logic unit** (simple ALUs).

The **P4 compiler first does dependency analysis on our code and maps it onto the various pipeline stages. The placement depends both on match and cation dependencies**.

Ternary content-addressable memory

The **matching logic** in our PISA is basically a **mix of SRAM and ternary content-addressable memory (TCAM)** used for lookup and other things.

A switch/router, to do routing, it needs to quickly compute the longest-prefix match of the packet in a set of rules. Since we run a pipeline, we want this match to be deterministic and always take a set amount of time in order to not stall the pipeline.

TCAM is a special type of associative memory that compares input data against the stored data and returns the address of matching data. They **produce the result of the query in a single clock cycle**. They are **very expensive**, so they need to be as few as possible.

TCAMs are called ternary because they use **ternary logic**: 0, 1, and _ (don't care).

The TCAM's buckets correspond to addresses in a RAM that contains the output port. This means that to match a rule we need:

- 1 TCAM access to get the address in RAM (constant time operation)
- 1 RAM access to get the output port (constant time operation)

A match is done in 2 memory accesses, which can be done in a single clock cycle.

Whether a TCAM matches the longest/shortest prefix is user-configurable and adds extra control logic to the memory.

TCAM is not cheap, so we cannot have a lot of it. We need the help of other data structure to efficiently create rules without using much memory.

Some data structures

Hash maps

You know how they work, here are some numbers (N elements for M cells):

1. Probability of collision: $P(N, M) = \prod_{i=1}^{N-1}(1 - \frac{i}{M})$

Bloom filters

A traditionally implemented set is not very well suited to our case: the output is deterministic, however the number of required operations cannot be known a-priori. What if **we can exchange the deterministic nature of the output for a deterministic time complexity?**

Traditional sets are very space inefficient: they tend to have a high probability of hash collisions for densly filled tables. To keep the conflicts down we need a very over-dimensionated table.

Let us start with a simple implementation: we have **an array of m bits and a hash function. If an element is in the set, we set the bit indexed by the hash to 1**. This structure, when queried for set-inclusion, will respond probabilistically; it can return:

- **true**-positives/negatives
- **false-positives** (different objects can have the same hash) with a probability of

$$1 - (1 - \frac{1}{m})^n$$

But **never false-negatives** (if a bit is 0, then all objects with said hash are not in the set).

We just described a **bloom filter with $k = 1$**. The k parameter **indicates the number of different hash functions that we will use on insertion**: each of those will calculate an index and set the corresponding bit to 1 in the array. To check for inclusion:

1. We calculate the k hashes
2. An element is considered in the set if all hashes correspond to a 1
3. An element is not in the set if at least one hash corresponds to a 0

The usage of k **hashes reduces the probability of false positives** to:

$$(1-(1-\frac{1}{m})^{kn})^k \approx (1-e^{-\frac{kn}{m}})^k$$

To find **how many hash functions we need, we can minimize** the above probability and it can be proven that the minimum k is $k = \frac{m}{n} \ln 2$.

To **enable element removal**, we can extend our filter to have one **integer per cell**, instead of one bit. **On insertion we increment** the bits corresponding to the hashes and **on removal we decrement them**. On query we check for > 0 and $= 0$. All our previous analysis still applies. This extension obviously **requires more memory, but also introduces the problem dimensioning the counters**: if a counter **overflows**, we incur the risk of introducing **false negatives**.

Invertible bloom lookup tables

Let us assume we have a counting bloom filter. How can we know which elements are stored inside the filter?

Instead of having only one array, we have **3**:

1. `count`: as before
2. `keysum`: the sum of all the keys mapped to a cell
3. `valuesum`: the sum of all the values mapped to a cell

Our operations become like this:

1. Query: as a normal counting bloom filter
2. Insertion: we update the filter, then sum the key and the value to the respective columns
3. Deletion: we update the filter, then subtract the key and the value from the respective columns

The **value of a key can be found if the entry** is associated with a `count` counter with value 1.

To **list all inserted elements** in a IBLT, we can:

1. **For all entries with count one**:
   - Remove them from the set and store the reconstructed values
2. **If there are no entries with count 1 we cannot do anything**

Sketches

Let us consider the problem of computing the frequencies of different objects in a stream. We could have a precise answer by using a counter for each type of element, however this requires a lot of memory especially if we have a lot of elements/element types.

Count-min sketches are a probabilistic data structure that serves as a frequency table of events in a stream of data. It trades off counting precision with performance.

It uses a **hash function to map events to frequencies**:

- We have d **arrays, with one hash per array of counters**
- Each array hash w **indices**

When an element is added, **for each hash, we increment the counter for the corresponding array and index**. **Collisions** between elements can cause **overcounting**.

When we **query** the number of occurrences of an element we return the **minimum of all the values corresponding to our element**.

It uses similar principles as counting bloom filter but it is **designed to have provable bounds for frequency queries**. It also **requires sub-linear space** (unlike hash tables) since the memory requirements do not grow linearly with the amount of data to be tracked. Due to how they work, the **counting error reduces for more frequent elements**.

Datacenter monitoring

At a very large scale, problems are difficult to debug: we might not even know were they might be (network, software, hardware etc.). Even figuring out the path of a packet is very difficult.

Network monitoring is the use of a system that constantly monitors a computer network for slow or failing components. We have different ways to monitor a network:

1. **From switches**: using their **features** (NetFlow, mirroring, SNMP) or **building new feature using dataplane programmability**
2. **From servers**: using **standard tools** (`netstat`, `tcpdump` or `traceroute`) or **ad-hoc tools**

Network monitoring using legacy switches

We would like to work around some problems like:

- **silent packet drops**: packets dropped but not reported by the culprit, it is due to software bugs or faulty hardware
- **silent blackhole**: a routing blackhole that does not show up in forwarding tables, caused by corrupted TCAMs.
- **inflated e2e latency**
- **loops from buggy middlebox routing**: due to a middlebox incorrectly identifying packets
- **load imbalance**
- **protocol bugs**

Problems would be easier if we could `traceroute` every packet, however at scale this is unfeasible. We might also need to **correlate packets over different hops**. Also the traces might not be enough if a problem is transient.

Let us analyze Microsoft's Everflow. The tree **main ideas** behind the method are:

1. **Match and mirror on switch**: **match** based on predefined rules **and mirror packets**. We mainly use matching rules for:
   - TCP's `SYN`, `FIN` and `RST`
   - A special debug bit in the header
   - All protocol traffic (BGP and others)
2. **Switch-based reshuffler**: traced traffic must be sent to different collectors as a single machine cannot cope with the amount of data
   - **We may use a virtual IP with a load balancer**
3. **Guided probing**: it allows to **inject any desired packet into the network and trace its behaviour**.

   It can be used to recover lost information with match&mirror and allows to check if a problem is transient or persistent. **Which bit can we use to signal it? The unused DSCP field and IPID fields in the IP header**: the first one to signal whether it is a flow of interest and the IPID to sample packets.

Everflow applications interact with a **central controller that knows the routing**. The controller gets as **input the operator request and the supposed network routing**; then it **initializes the rules on switches, configures the analyzers and sets the appropriate debug bit into probes**. After the configuration, the **mirrored traffic will be sent to the analyzers where they were heuristically checked and stored if problematic**.

Takeaways: although powerful, it is still **limited by the capabilities exposed by the switches**.

Network monitoring using dataplane programmability

An important trade-off to consider is where to put the monitoring overhead:

1. Collectors have limited bandwidth and storage
2. Switches have limited memory and processing time

FlowRadar

Let us analyze FlowRadar to find a way to minimize the load between the two. **FlowRadar allows us co compute packet counts across different flows**.

Architecture:

- **Each switch maintains a fast and efficient data structure for half-baked per-flow counter and sends periodic reports to collectors**.
  - We maintain a **tree column table**:
    1. `FlowXOR`:: the XOR of all the flows mapped to a bin
    2. `FlowCount`: the number of flows mapped to a bin
    3. `PacketCount`: the number of packets of all the flows mapped to a bin

  If a **flow is new, everything is updated, otherwise only the packet count**. We implement the following **filter using an invertible bloom filter**.
- The **collectors correlate network wide info to extract per-flow counters**. The work is divided into **two stages**:
  1. **Single decode**: **compute the number of packets by inverting the flow filter on a per-switch basis**:
     1. We find a cell with one flow (pure cell)
     2. Remove the flow from all the cells
     3. Continue removing all flows until have no more pure flows.
  2. **Network decode**: **we solve eventual stalls in the single decode by leveraging network wide info (info from other switches)**.

     We use merge tables into a single mega-switch table and use it in our algorithm. To find which switch processed what flow we can query its bloom filter.

In-band Network Telemetry

It is a **framework designed to allow the collection and report of network state, by the data plane**. In the INT architectural model, **packets may contain header fields that are interpreted as telemetry instructions by network devices**. These **instructions tell an INT-capable device what state to collect**. We can see this as a much more powerful version of Microsoft's Everflow.

We can run INT into **three modes**:

1. `XD`: **Each nodes exports metadata** based on watchlist configuration
   - Good: no packet modification
   - Bad: pressure on collectors, query based on switch configuration
2. `MX`: **Embed only instructions in the packet. Each node exports metadata**.
   - Good: query is packet dependent
   - Bad: pressure on collectors, modified packets
3. `MD`: **Embed instruction and metadata, export at the sink node**.
   - Good: query is packet dependent, no pressure on collectors
   - Bad: impact on packet MTU (the switch will eat space into what is usable by the application), packets need to be modified

Datacenter load balancing

Layer 3 load balancing

Many distributed applications run in a datacenter, these generate traffic within the datacenter with very variable characteristics. To best support distributed applications we need a high bisection bandwidth. Lots of paths available from one rack to the other, so **how do we best utilize network resources**? This is the goal of L3 load balancing. This is hard because flows come and go and we have a mix of short and long flows.

A **simple technique** is to do **packet spraying**: we "spray" the packets to all the possible paths.

- Pros: **traffic is well spread** (even with heavy flows)
- Cons: **interacts very badly with TCP**:
  - Packet reordering can trigger duplicated ACKs
  - Sender will reduce the sending rate as it thinks packets are lost

This solution is **simple but very bad**. We will see some better ones.

Equal Cost Multi Path (ECMP)

We use **per-flow load balancing**. This can be **implemented with a hash function**: `port.output = hash(packet.tuple)`; the tuple of fields used is switch-implementation-dependant.

Basically **all packets belonging to a flow (i.e. packets that have the same tuple hash) are routed to the same path**.

- Pros: **a flow follows a single path**
- Cons:
  - The **performance depends on traffic characteristics** (hash collisions)
  - **No optimal load balancing** (on average it could be optimal)

The main **problem** with ECMP is that it **is static and completely oblivious to link utilization**. More over, it **can cause long-term flow collisions due to hash collisions**. It has been proven that ECMP on average wastes 61% of the bisection bandwidth.

Hedera

It is a solution proposed to **improve and complement ECMP**. It is **built to work on top of a SDN network and tested with the OpenFlow protocol**.

Given a dynamic traffic matrix of flow demands, how do you find paths that maximize network bisection bandwidth?

> A traffic matrix is a two-dimensional matrix with its ij-th element (t_{ij}) determining the amount of traffic sourcing from a node i and exiting node j

Assuming we have OpenFlow switches and a centralized controller, we can **execute the following loop forever**:

1. **Pull stats** from the switches and **detect large flows**
2. **Compute the demands**
3. **Compute the placement**
4. **Place the flows**

We **schedule only elephant flows since ECMP is enough to deal with shorter flows**.

Flow rates are a poor indicator of flow demand: the network could be the bottleneck. **Hedera's approach to computing flow demands is assuming no bottleneck in the network, we compute each flow's demands by considering shares of sender and receiver flows**.

For **placing flows**, we have **two policies**:

1. **Global first-fit**: when a new flow is detected, we linearly search all possible paths from source to a destination. We place the flow on the first path whose component links can fit that flow.
2. **Simulated annealing**: probabilistically search for good solutions that maximize bisection bandwidth

In **global first-fit a large flow can be rerouted upon detection and is essentially pinned to its reserved links. Simulated annealing, on the other hand, waits for the next scheduling tick and uses previously computed flow placements to optimize the current placement**.

Problems with this approach:

1. **Dynamic workloads can generate large-flow-turnover faster than the control loop**, meaning the controller will be continuously chasing the traffic matrix.

2. **Can it scale to really large datacenters**?

In-network load-balancing (HULA)

If the problem is the centralized controller, **can we get rid of it and implement load balancing back in the switches (as ECMP was doing)?** This **makes sense because datacenter traffic is very bursty and upredictable**, meaning that there is not much time available for the control loop.

The problem with ECMP is that it is static… **What if as a new flow arrives we assign it the least congested port? Why are we working per-flow and not per-packet**? Per-packet load balancing would be ideal but introduce packet-reordering which is bad for TCP; per-flow load balancing is fine but too coarse-grained to fully utilize the network.

We can **divide a flow into flowlets: bursts of packets from a flow that are separated by large enough gaps. If the gap is large enough then it is possible to route the two flowlets to different paths without risking reordering**.

Putting it all together we **obtain a congestion-aware load balancing solution with flowlet switching, built to work on P4 programmable switches**: HULA (Hop-by-hop Utilization-aware Load-balancing)! The challenges we face are:

1. If we want to create a congestion-aware algorithm, how to keep track scalably of all the possible paths?
2. How to do this continuously (in a proactive manner)?
3. How to implement such an algorithm given the constraints of programmable switches?

Hula switches exchange probes to propagate path utilization (allowing continuous monitoring of network status). **Each switch remembers only the best next hop, removing scalability/topology problems. We will use flowlets to split elephant flows into smaller chunks**.

A HULA probe is a minimum-sized IP packet, including the HULA header:

```
+---------------------------------+
| Standard IP Header              |
+---------------+-----------------+
| HULA switch ID | HULA path util |
+---------------+-----------------+
```

Some questions:

1. What happens on **link failure**?
   - **Switches update their table only considering probes they receive**. The probes act as a sort of **heartbeat**.
2. Shall we **run the probes more or less frequently**?
   - **Frequent probes generate more overhead but provide better network visibility**
3. What's the **right flowlet timeout**?
   - **It depends on the network RTT**

Layer 4 load balancing

In a traditional cloud load balancing architecture, we have different levels of load-balancing: Network (L3), Transport (L4) and Application (L5). Application-level load-balancing is usually tenant-written software, while network load-balancing is usually managed by the datacenter owner and is hardware-based. L4 is usually datacenter-owned but software implemented.

Similarly to L3 load balancing, our goal is **finding a way to spread requests over multiple servers. We have three possible approaches**:

1. Load balancer **terminates TCP and opens own connection to servers**
   - Problem: **very intensive** since we have one connection for each external connection and one for each internal server
2. **NAT approach**: load balancer replaces service VIP with server's actual IP
   - Problem: we **force each stream to pass into software**

3. **DSR** (Direct Server Return): **servers bind both the VIP and the dedicated IP, load balancer replaces the destination MAC. Server sees the client IP and responds directly, i.e. exiting packets do not pass through load balancing**.
    - **Greater scalability**, particularly for asymmetric bandwidth

What **policy** do we use to equally share resources to the servers?

- **Round robin**: the load balancer has a counter, when a new request arrives, assign it to the server indexed by the counter and increase the counter
    - We **break-up flows**: requests may not fit into one packet and packets belonging to the same request must hit the same instance
    - **Processing request time may vary**: some server may be slower or some requests are more complex

Let us **focus on correctly spreading requests**. To tackle this issue **we must implement a notion of persistence** for load balancers: **packets belonging to the same flow are forwarded to the same server**; therefore we must guarantee packets belonging to the same TCP connections hit the same server even in the face of topological changes. We need a **uniform deterministic load-balancing function**:

- Uniform: we need to spread loads across servers
- Deterministic: we need to guarantee connection affinity

Options:

1. Keep **per-flow state** at the load balancer, a TCP/IP tuple identifies a connection
    - Stateful solution that might require a **lot of memory**
2. Apply a **hash function** h on the packet header. This allows us to map arbitrary size data into data of fixed size.
    - **Pros**: **zero state**, packets of the same flow will hit the same server (not necessarily of the same user)
    - **Cons**: **does not resist server-pool updates**
3. **Consistent hashing**: **every server is responsible for the hashing space until its predecessor**.
    - **Removing servers might lead to imbalances**: a solution might be mapping each server to multiple points on the circle. When adding a new node the load will be balanced.
    - On server addition, we still have a problem: **how do we know some of the connections in the new server's range were pre-existing and shall be forwarded to the previous server**? We would need to **reintroduce some statefulness**.
    - The above might not be enough: what if a **load balancer itself goes down**? We might **replicate state** to different load balancers (increased latency).

Cheetah

We want a stateless load-balancer, but we do not want all problems associated with consistent hashing.

1. A **new request from a user hits the load balancer and a server is selected**
2. On the **way back, a cookie is inserted** in the reply with the originating server ID
3. **New packets of the same flow need to just insert the cookie**

To remove the possibility of DoS a server by forging packets **we can encrypt the server ID by XORing the server ID with a secret hash of the connection ID**.

As a **flow-assigning strategy we can implement each strategy** (even round-robin, although it not always the best option). We can **also implement DSR**, but we **need to change the server's network stack** to compute the cookie. Cheetah **does not require changes to the client side** since we can use **standard TCP features** to store the cookie (timestamp echo).

Fastly's solution (faild)

This is a solution for the edge. **Requests enter a specific network through Points of Presence** (PoPs); **requests that cannot be fulfilled by a PoP are routed through an owned backbone network to a datacenter**. **PoPs**, then, become **small datacenters**: this means that they need to be

1. High performance

2. Do not be a single point of failure
3. Absorb DDoS attacks
4. Do not cause service disruption on maintenance

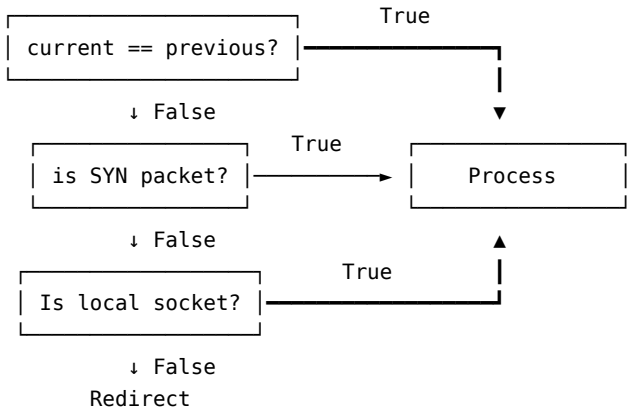
We need a load balancing architecture which is:

- Efficient: no dedicated hardware and no much space available
- Resilient: anything that maintains state is easy to DDoS
- Graceful: ability to change services without disrupting flows

`faild` **maps the VIP to a static set of "virtual" next hops** (to avoid ECMP rehashing). The **forwarding is controlled using the ARP table**; **on drain we update the affected IPs and balance the virtual hops across the available servers**. This is just consistent hashing with extra steps, however **since we are playing with fake MACS we can**:

- Keep a **mapping history**:
  - `xx:xx:xx:xx:a:b`: `a` is the ID of the current host, `b` is the ID of the previous host.
  - Keeping only previous and current target conveys enough information to down to the host

Each host can inspect the destination MAC of the packet and execute the following algorithm to decide what to do with the packet:



This effectively means that **packets filtered through the host are only accepted if they belong to a new connection, or if they match a local TCP socket**.

End host networking

Deploying new hardware to end-hosts usually requires a lot of investment, deploying only the new functionality is much easier.

From a high level perspective, **every packet has to traverse**:

1. the **NIC**
2. **PCIe**
3. NIC **driver**
4. **Kernel**
5. **Application**

The hardware land

The **kernel allocates memory for storing the received packets from the NIC** (`Rx` buffers). The **driver populates** the `Rx` queue in the NIC **with pointers to the `Rx` buffers** (`Rx` descriptors). The size of `Rx` buffers is configurable.

When a packet arrives at the NIC, it is **first stored in its local memory**. Then the **NIC fetches one descriptor from the `Rx` queue** so that it will know where to transfer the packet in the host memory. The **NIC starts a DMA transaction over PCIe to move the packet from NIC to host memory**. Finally

the **NIC generates an Interrupt ReQuest (IRQ) to inform the driver of new data** to be processed. A **CPU core will take the IRQ** and the processing on the host starts.

This is a very simplified view into the matter: **high speed transfers can generate very frequent interrupts**, reducing performance. Modern NICs provide **ways to mitigate the number of interrupts sent** from HW to the CPU:

- **Interrupt coalescing**: delay interrupts and process multiple events at once with a compromise between reaction time and overhead
- **Polling**: the CPU continuously checks if the device has anything to send. It relies on busy-polling loop potentially wasting resources
- **NAPI**: adaptively switching between the previous two according to the current workload

Modern NICs provide **multiple hardware queues**. We can **assign each queue to a CPU core** and load balance the processing. This is called **Receive Side Steering** which assign flows-to-queues **based on hash**. The **problem**? Like with all hash-based methods, **hash imbalances**. They also **provide some "standard" functionality that the operating system can accelerate in-hardware**:

1. **Checksum calculation**
2. TCP Segmentation Offload (**TSO**) and UDP Fragmentation Offload (**UFO**): NIC handles segmentation/fragmentation
3. Large Receive Offload (**LRO**): NIC re-segment incoming packets

Modern CPUs provide the **possibility to directly store the packet in L3 cache** (LLC). The CPU core does not have anymore to move data from DRAM to cache when processing the packet. The **problem**? New incoming packets repeatedly evict not-yet-processed packets from the LLC (**leaky DMA**).

All the operations discussed so far **require PCIe transactions**. When you DMA something (e.g., write X to host at address Y), the DMA engine **breaks the request in multiple PCIe Memory Write packets** (Transaction Layer Packets). **PCIe is almost like a network protocol with packets (TLPs), headers, MTU, flow control, addressing etc...**

The kernel world

The kernel relies on a **specific data structure to handle the packet during its journey** towards the application logic: this is called **Socket Buffer** (`sk_buff` or `skb`). The **driver allocates a socket buffer for each received packet**. They are **organized in circular lists** to speed up certain operations.

Some pseudo-C of how the `skb` is laid out (in reality it stores much more information).

```
struct skb_list_elem {
  skb_list_elem* next, prev;
}

struct skb {
  mem_ptr head; // Start of buffer
  mem_ptr data; // Start of packet
  mem_ptr tail; // End of packet
  mem_ptr end;  // End of buffer
}
```

The **size of the overall buffer is bigger than that of the packet so that new headers can be added without the need to allocate new structures**.

How things move up the stack:

1. The **kernel allocates `skb`s** for each packet
2. The **Generic Receive Offload (GRO) module attempts to reduce the number of `skb`s by merging them** (this is why we have them stored in a circular manner). This operation is the **software counterpart of LRO**.

3. **`netfilter`** is a framework that offers various functions and operations for **packet filtering, network address translation (NAT), port translation and connection tracking** (`iptables` uses `netfilter` to implement its policies)
4. The **TPC/IP module** processes all transport-layer stuff
5. The **`skb`s are appended to the application's socket's receive queue**. The **application performs data copy of the payload** in the `skbs` in the socket receive queue to the userspace buffer (the kernel does not do any memory copying)

`skb` allocation and processing is expensive. The TCP/IP stack is also very expensive. If we employ all in-hardware optimizations, the heaviest operation becomes data copy between userspace and kernelspace.

Improving software performance

DPDK

First option for improving performance is **removing directly the kernel and make the NIC write directly in userspace memory**. However, by bypassing the kernel **we lose all the features that the kernel gives us** (TCP/IP, NATting, firewalling etc…). This means that every application needs to:

1. Implement all the network stack from the metal up
2. Have total ownership of the DPDK port (we can share resources only by virtualization techniques like SR-IOV)

Programmable NICs

What if you could **programmatically change the behavior of the NIC**? We might want to do this for **different reasons**. **One is performance, the other is that we can separate datacenter functionality from customer functionality**. As a datacenter operator, **we put all our functionality in the hardware, ensuring that no one can tamper with how the datacenter works**.

eBPF

We could **improve the kernel by**:

- **Creating an ad-hoc kernel module**, however this methodology **requires that we maintain a custom kernel or need to upstream the changes**
  - see OVS
- **eBPF** enables **dynamic code injection and execution in the kernel** while providing **hard safety guarantees** to preserve system integrity
  - In mainline linux
  - **No need for additional modules, neither to recompile the kernel**

Feature overview:

1. **Runtime bytecode injection**: eBPF programs can be dynamically created and injected in the kernel at run-time
2. **Safeness**: achieved via a **verifier that imposes hard rules on what the program can do** (no invalid accesses, bound on program complexity and loops). This means that we cannot push arbitrary programs in the kernel.
3. **Efficiency**: **consumes very little resources**, plus it runs in kernel space so **userland-data-copying is avoided**.
4. **Kernel event reaction**: eBPF code is hooked to a kernel event. When fired, our code (associated to an event handler) is executed. We can use the **following hook points**:
   - `XDP`: programs run at **driver level**, this means we do not have netfilter or TCP/IP processing. This means that **we can only deal with stateless protocols without breaking things** above us.
   - `AF_XDP`: packets **are sent directly to the socket skipping all steps after the driver**. It is **equivalent to using DPDK**.
   - `TC`: hooks at **netfilter level**, useful for traffic-control tasks

- `SK_SKB`: hooks at the **socket level**, after TCP/IP processing. Useful for implementing **"application-acceleration" at the kernel level** since we can do similar processing without copying data around

We can **chain different eBPF programs via tail calls**. They are simply optimized into a long jump, reusing stack frames. Programs are still verified independently and only programs of the same type can be tail called.

eBPF can call a selected number of kernel functions using helpers, which function like P4 `externs`.

eBPF uses a pre-formatted memory layout. Data is accessed using maps. Maps can also be used to share data between eBPF programs and userspace or different eBPF programs. Interaction with maps is done through helpers.

Congestion control

Recap on how TCP congestion control works:

- Three way handshake (`syn-syn ack-ack`) (connection oriented)
- Data is split into segments
- Byte oriented
- Number of segments sent is dependent on the TCP window
- The TCP window is increased using two policies:
  1. Slow start: $W = W + 1$ per `ack` until `W <= SSTHRESH`
  2. Congestion avoidance: $W = W + \frac{1}{W}$ per `ack`
- `acks` can be lost and the un-`acked` are resent after a timer (`RTO`) and the congestion window is reduced
- Packets can be lost, subsequent packets are `acked` with the last sequence number received; retransmission works similarly to lost `acks`
  - Fast retransmit: after three identical `ack`, trigger retransmission immediately without waiting for `RTO`
- `acks` can be delayed and sent cumulatively after a timeout (smaller than `RTO`)
- Packets can be reordered
  - `acks` for out-of-order packets carry the same `seq` as the last in-order packet, easily triggering fast-retransmit
  - TCP behaves badly for out-of-order packets

TCP does not adapt well to datacenter networking requirements. Many large scale applications are built on **the partition/aggregate design pattern:** basically a hierarchy where each layer manages the layers below (**worker-aggregators**).

Problems:

1. **Incast**: if **many flows converge on the same switch** interface over a short period of time, **they might saturate the buffer and induce packet loss**. Packet loss **causes a reduction of the TCP window**, meaning we go slower.

   We can **de-synchronise the workers** by deliberately delaying their answer randomly (**jittering**). This, while avoiding incast and **improving the 99-th percentile, increases the median response time** since we are bounding ourself to a delay.

   Why don't we **reduce the `RTO`**? Reducing the `RTO` makes us able to respond to packet losses more quickly.

2. **Queue buildup**: **short and long flows compete for space in the output queue** buffer of a switch:

   - **Short flows** need **low latency**, meaning empty queues
   - **Long flows** need **high-throughput** and big congestion windows

   We have **two** possible outcomes:

   - **Packet loss** (incast)
   - **Queue buildup**: short flows experience increased latency as they're in queue behind packets from large flows

   Reducing the `RTO` does not help since resending packets means that new resent packets get queued behind big flows.

3. **Buffer pressure**: in switches, **buffer space is shared between ports**. **Long flows build up queues** and since buffer space is a shared resource, there is **less space for other flows**.

Datacenter needs:

1. **High burst** tolerance (solves incast)
2. **Low latency** (for short flows)
3. **High throughput** (for large flows)

The **challenge is achieving these three things together** since the interests of some conflict with those of others (e.g. deep buffers guarantee (1) and (3) but make (2) worse).

Explicit Congestion Notification (ECN)

Is an **extension of TCP** that **allows end to end notification of network congestion without dropping packets**. It **requires support at the IP layer**. It signals impending congestion of the link.

Basically: if the **ECN bit is set** in the IP header, then the **path is congested**. The **receiver will `ack`** the data and piggyback the ECN bit, while the **sender will react by halving the TCP window**.

DCTCP

Built to work with legacy switches on top of commodity ECN features. The idea is to **extract feedback from single-bit streams of ECN marks and react in proportion to the extent of the congestion**.

The **only difference** between DCTCP and TCP is **how we convey information about congestion back to the sender**: DCTCP **tries to accurately convey the exact sequence of marked packets back to the sender using only 1-bit congestion information**. We cannot `ack` every packet, but we can **exploit delayed `ack`s to implement something like a state machine in the receiver**:

1. If we receive a lot of non-ECN packets we send only a cumulative-`ack`
2. As soon we receive a packet with the ECN bit set, we flush the `ECN=0` `ack` and then send the `ack` for the `ECN=1` packet
3. Then we continue sending delayed `ack`s but with `ECN=1`
4. If we receive an `ack` with `ECN=0` we flush the ack for `ECN=1` and start delay-`ack`ing for `ECN=0`

TCP always halves the window size in response to a marked `ack`, **DCTCP starts reducing it gently**. When **high congestion** is found, **DCTCP behaves like TCP**, however with very low congestion, DCTCP reduced the window very slowly (for the formulas for computing the window see the slides or the DCTCP paper).

DCTCP also **uses big buffers** so they can be almost always empty and guarantee high burst tolerance.

DCTCP works since we can have:

1. Low latency: by having **low buffer occupancy**
2. High throughput: by having **smooth window adjustments**
3. High burst tolerance:
   - By **keeping** the **buffers mostly** empty
   - By using **aggressive marking**, so **source react** before we start dropping packets

DCTCP however is not optimal: latency-sensitive parallel **short flows can get stuck waiting behind resource-intensive, long flows in switch queues**. We need to **handle flow prioritization at the switch level**.

pFabric

Decouples rate control and flow scheduling enabling a **new transport protocol** that provides near-optimal flow completion times. **Needs specific functions at switches**.

pFabric's **essential idea**:

1. **Packets** carry a single **priority number**

2. **pFabric switches** have **very small buffers** and **process only high priority** packets while dropping lower priority ones
3. **pFabric hosts send/retransmit very aggressively with minimal rate control**
4. We **do not care about fairness**, we need to **complete flows as quickly as possible** (i.e. **meeting overall computing objectives**)
5. The **network fabric should be designed** to schedule flows **to maximize application level objectives**

Let us **abstract** a datacenter as a **gigantic switch**: with these abstraction we can see that:

1. The objective is minimizing flow-completion-time
2. Datacenter transport is just flow-scheduling on a giant switch
3. Ingress and Egress are the capacity constraints

The **optimal algorithm** for **minimizing average FCT** when scheduling over a **single link** is the **Shortest Remaining Processing Time** (SRPT) which **schedules the flow that has the least work remaining**. We are not scheduling over a single link, however **prioritizing small flows over large flows end-to-end across the fabric can provide near-ideal average FCT**.

The first roadblock with our policy is **dealing with starvation**: blindly dropping low priority packets means that those flows will never complete as long as there are higher priority ones. The **solution** is not blindly sending high-priority packets, but **dequeueing only the earliest packets from the flow with the higher priority**.

pFabric's rate control is very simple, but if we make it too simple it can lead to wasted resources (packet traverses almost all hops only to be dropped on the last). We need to **prevent congestion collapse** due to colliding big flows on hops. The **solution** is a **diet-TCP protocol**:

1. **Start at line rate** (use an initial window size equal to the BDP of the link)
2. There are **no fast-retransmits** or any other similar mechanisms. **Packet drops are only detected by timeouts** (fixed and small, e.g., 3x the fabric RTT).
3. **Upon a timeout, the flow enters into slow start and `SSTHRESH` is set to half** the window size before the timeout occurred.

Why it works?

- **When a packet is dropped**, it has the **lowest priority among all buffered** packets. **Even if it were not dropped, its "turn" would not be until at least all the other buffered packets have left the switch**
- **Consequence** of the previous point: a **packet can safely be dropped if rate control is aggressive** and **ensures** that it **retransmits the packet before all the existing packets depart the switch**
  - This can easily be **achieved** if the **buffer size is at least one BDP** and hence **takes more than a RTT to drain**, providing the end-host enough time to detect and retransmit dropped packets

Seminars

SD-Fabric

The first generation was more focused on achieving scalability and fault-tolerance for the traditional L2-L3 fabric. It is built by adhering to classic SDN principles: control plane out-of-box, no distributed protocols and a high level API for apps (the fabric controller was one of the applications). It is built on white-boxes and merchant silicon with minimal switch software (controlled via OpenFlow). It was put into production at Comcast with great savings on deployment and operational costs by consolidating servers and offering more scalability headroom.

Eventually, the system migrated from OpenFlow to P4Runtime and a fully-programmable leaf-spine fabric and gRPC based APIs (like gNMI (management)and gNOI (operations)). Multi-tenancy was introduced and many more advanced features directly on the switch in P4, including INT. Having INT enabled data report collection and analysis by operators to optimize/fix problems by pushing new configurations using the SDN controller.

Some advanced features are possible only thanks to centralized management. This management can scale thanks to the use of DevOps tools and practices like:

1. Extensible model-based config: from switch ports to policies
2. Continuous software updates with CI/CD automation:
    - New code gets pushed to a git repo through a MR and then automatically integrated by a CI pipeline after automated tests and code review:
        - Pre-merge checks: License check, various tests and the peer review
        - Post-merge checks: Validation tests and other tests to avoid regressions

Pre-merge

Unit tests are specifically for the control plane. Some SD-fabric specific tests are dataplane tests:

1. We control a virtual switch with P4Runtime and start injecting hand-crafted packets, testing the packet output
2. First we tests on 100% virtual models (like V1) and then on emulated real architectures (Tofino)

E2E tests (sanity tests) are done by emulating a real network using mininet (all in software for speed). The goal of these tests is not test every possible functionality, but verify that no major defect/regression have been introduced.

There are also some manually-triggerable tests that are more long-running (like tests requiring physical hardware) so they are triggered only in specific circumstances.

Post-merge

After merging the PR, all tests are re-run to verify that the tip of main functions as expected and then artifacts are published.

Nightly tests, which are more long-running and advanced tests (even on real hardware) are run to further validate the new features and ensure that there are no new regressions. Other periodic tests are:

1. Scale tests: ensure that the new features do not impede and lower performance
2. Soak tests: validate the system behaviour under production use

Releases

After everything has been validate, we can release a version by tagging a commit in a repo in order to have a reproducible and deployable version. Semantic versioning is used.

OVS

OVS is a software network switch. It is built to be:

1. Sophisticated: Biased toward difficult cases.
2. Fast: Especially in those difficult cases.
3. Automated: All features remotely controllable.
4. Software: Only requires ordinary NICs.

It is programmed using OpenFlow to program flow tables, which are basically glorified if-else with conditions and actions.

The hard part is not implementing all of this, but making it fast.

Keys to speed in any software switch:

1. Fast packet I/O
    - It is important, and orthogonal to switch architecture
    - Just use the best one (the OVS module, eBPF or DPDK)
2. Low per-packet overhead
    - A software switch are structured in a series of stages, which are independent and can be parallelized. OVS adds a parsers to the pipeline, which increases overhead
3. Low per-packet processing cost
    - Each stage in the pipeline needs to be cheap
    - They usually are just a function call, or nothing at all

- OVS stages are general-purpose flow-table lookups, which are expensive on general purpose hardware

One way to make OVS fast in spite of all the overhead it requires is caching.

The simplest way to add a cache is to add some to each stage. But this is done by everything, so OVS still has the performance disadvantage. A better way is to cache the entire pipeline, meaning a single cache hit skips all the stages; this cannot be done by other software switches.

For small pipelines, simple caching is faster, however it scales badly with big pipelines. Compound caching on the other hand scales better for deeper pipelines (which is the use-case for which OVS is optimized), surpassing simple caching.

Caches have a long history with networking (e.g. ARP or route caches). However they are also infamous:

- Hit rate depends on traffic, leading to unpredictable performance
- Invalidation can be tricky to handle

DBSP

DBSP is a system for incremental view maintenance (recall what views are in e.g. SQL). Incremental view update is a technique that allows a database to update the view without recomputing every entry. DBSP does exactly that: it takes a change to a table and translates it into a change to the various views.

Let us define a new relational operator called "network classifier join" that takes as input a flow table and a set of packet headers and outputs the best match for each header in the table. With DBSP we can also apply this operators to deltas (Δ flow tables etc…).

The ability to work with deltas means that we can incrementally update the cache for each packet that doesn't hit/network change without needing a full rebuild.

Offloading more packet processing to hardware

As network speeds increase, core counts remain roughly the same. This means that software would need to process more packets with with the same number of cores available.

In OVS we have two paths:

1. Slow path: taken for the first packet in a connection
2. Fast path: taken for subsequent packets

Originally, the slow path run through userspace and fast patch in kernel, however the kernel does not meet speed demands anymore. Now the fast path is:

1. Passed through a DPDK userspace to speed up packet processing
   - Uses more CPU, so not exactly useful
2. Offloaded to a NIC asic (smart NICs)

Now only userspace remains slow. So eventually smart NICs integrated some general purpose CPU controller with offload capabilities, meaning we can offload the slower "userspace" onto it. The problem is that NIC cpus are generally slower than the host one, bringing us to square 1.

The slow path does 3 expensive things:

1. Flow setup
2. Flow cache revalidation
3. Control protocols

If we can move any of them to hardware (a domain specific architecture), we could speed the path very much.

Optics for the cloud

To support high speed transfers for long distances (> 2m), data centers use high-speed optical transceivers that convert back and forth optic signal to electric signals. This electrical conversion + the general end of Moore's law due to slow speed improvements w.r.t. power consumption prompted the research of new ways to keep up with the speed increase due to the increasing use of optics.

One way to keep up is the use of optical switches controlled by tunable lasers, which are basically mirrors that do switch on a wavelength basis. This has several advantages:

1. Future scaling in the post-Moore's law era (bandwidth agnostic)
2. Ultra-low and predictable latency (no buffers in the core)
3. Sustainability: very low power (no transceivers or switches in the core)

However this design requires innovation across the whole cloud stack since most assumption of networking would change: no more buffers (so no TCP for example), predictable latency and the possibility for a synchronous system.

The trend of using optics is not limited to networking since other components are also facing end-of-Moore's-law-problems:

1. Storage: we have the need for ultra-low power, ultra-high density storage for archiving very "cold" data
   - Optical storage can provide solutions
2. Compute: we are reaching the limits of how small a transistor can be and hitting the wall of quantum mechanics
   - Optical CPUs can provide solutions for this

Any of this changes will, however, require a holistic approach to the data-center stack: we do not need to optimize components in isolation, but to imagine the datacenter as a giant computer.

Considerations for end-host networking

Smart NICs

Ethernet traffic is usually 25 to 100Gbps. Data centers are moving rapidly to 200-400Gbps. The minimum frame size is of 64B, maximum is 1518B (with Jumbo frames up to 9000B). Datacenter traffic is typically encapsulated, usually with Geneve/VXLAN (L2 frame in UDP) with a min size of 54+B.

Considering a simple 1GHz NIC processor that executes 1 instruction per nanosecond and a 100Gbps with 256B frames, we would have 22 instructions at disposal per frame. We can do more stuff in the same amount of time with the help of multi-cores and some specialized hardware for some offload.

Memory bandwidth is also getting tight, but less than compute power (memory bandwidth is still much bigger than peak network bandwidth). Memory access latency is however very tight: at 100Gbps-256B frames we would not have enough time to even access L3 on state of the art hardware. To hide latency we need to heavy multi-core and, to reduce even further, hardware accelerators.

Let say that accessing a simple Hashtable requires 460ns per packet of memory latency and 100 instructions. With a simple 1Ghz processor:

1. Single core: we can handle 1.7Mpps, which is line rate for 7000B@100Gbps
   - Pretty far from the max Ethernet frame sizes
2. 16 cores: we can handle 27.2Mpps, which is line rate for ~512@100Gbps
   - Multicore hides memory latency
   - Multicore introduces packet-reordering problems

This quick math is an oversimplification since in reality a packet would require much more computation/lookups.

Some common acceleration used by commercial programmable NICs is:

1. Fast(er) packet memory
2. Programmable classification engines
3. Lookup engines (near memory): like TCAM
4. Queue engines
5. Stats engines (fire and forget stats updates)
6. Inline and look aside crypto
7. Compression engines

PCIe

PCIe is the defacto standard to connects high-performance IO cards. It is implemented on the host side in the Root complex. PCIe devices transfer data to/from host memory via DMA. DMA engines on each device translate requests like "Write these 1500 bytes to host address 0x1234" into multiple PCIe Memory Write (MWr) "packets". PCIe is almost like a network protocol with packets (TLPs), headers, MTU (MPS), flow control, addressing and switching (and NAT). Each TLP has 3 layers worth of headers and a max length of 256B (typically).

PCIe adds overhead. The various generations mostly define encoding/speed on the wire while having multiple lanes increases bandwidth. For each Memory Write packet, we have at least 10% header overhead. Doing some quick math+graphs, we can see that with PCIe gen4x8 we can barely keep up with 100Gbps Ethernet for longer MTUs.

The overhead, however, increases since NICs do not simply write/read data, but handle descriptor rings for buffers, which requires more transactions. This means that we go well-below 100Gbps Ethernet (only for very large MTUs we come close to line rate) even with a well-optimized driver or DPDK when using Gen4x8.

PCIe also adds latency overhead, which needs to be accounted for. DMA engines in NIC need to manage many in-flight DMA packets to hide this latency. Latency overhead also increases due to the need to perform address translation (Bandwidth can drop by 20% when thrashing the TLB).

SmartNICs have evolved towards including more functionality than just networking, becoming more generic Data Processing Units. Some functionality implemented is:

1. Network attached/local storage
2. RDMA, which is DMA to the memory of a different computer
3. Control plane for virtualised servers